

지도교수: 이상준

2026.04.16

ROI 기반 MSA 선택적 전환 결정 기법

ROI Based MSA Selective Transition Decision Technique

저자: 소프트웨어공학과 4102410005 하지연





목차

I 연구 배경 및 목적

II 관련연구 및 문제점

III ROI 기반 MSA 전환 결정 기법

IV 향후 연구 일정



연구 배경 및 목적

모놀리식 구조의 한계 및 마이크로서비스 아키텍처(MSA) 전환의 필요성

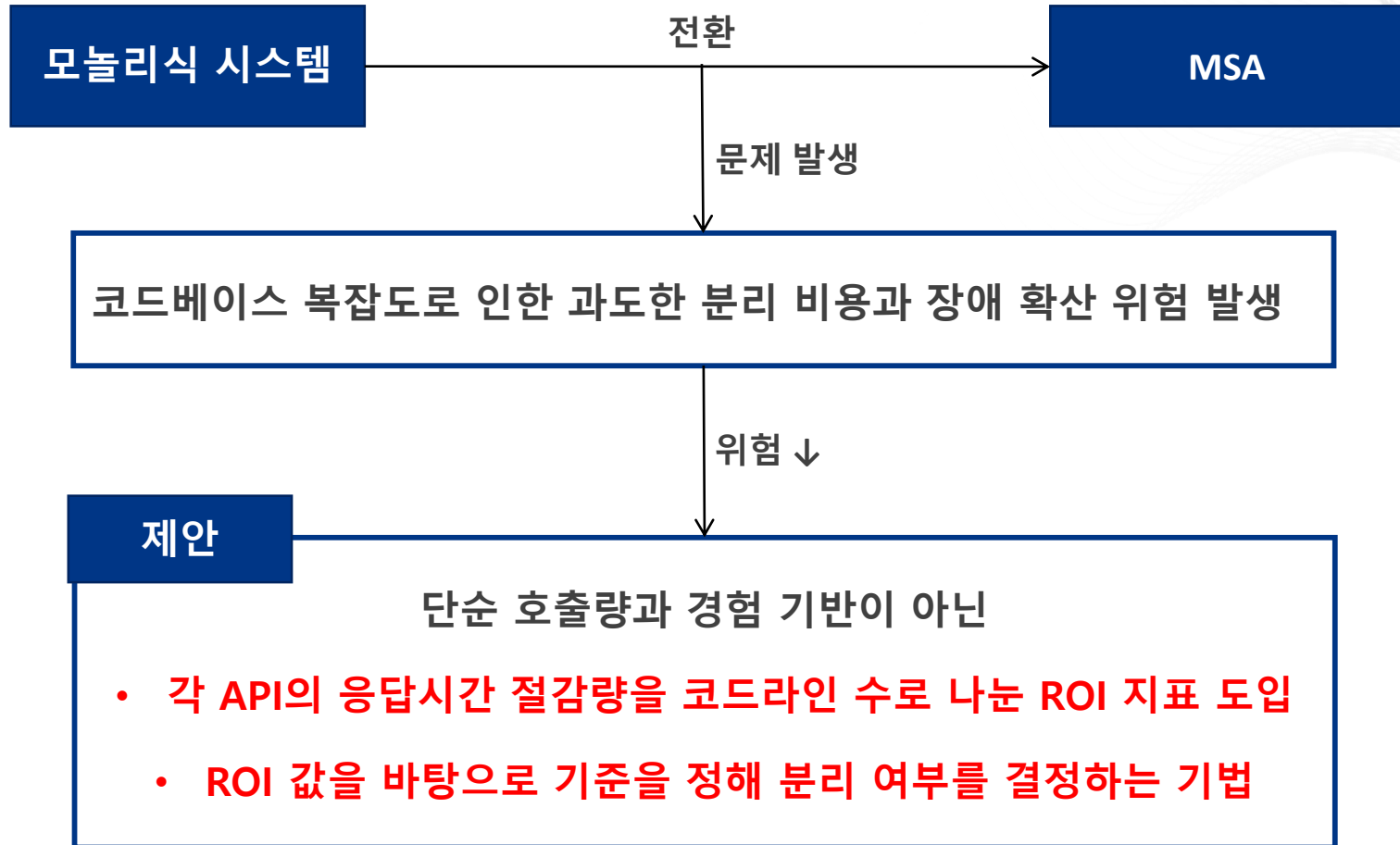
모놀리식 아키텍처	코드베이스가 커질수록 수정과 배포에 시간이 오래 걸리고 장애 확산 위험이 높음
마이크로서비스 아키텍처	서비스 단위로 독립 배포와 확장이 가능해 민첩성과 안정성 모두 개선 가능



문제점

- 전환 우선순위를 무작위나 경험에만 의존 시, 비용은 높는데 성능 개선이 적은 기능을 분리할 위험이 있음
- 정량적 근거 없이 기능을 선택하면 전환 효율을 최적화하기 어려움

연구 배경 및 목적



도메인 중심 설계(Domain-Driven Design, DDD)

- 복잡한 비즈니스 도메인을 효과적으로 모델링하기 위한 소프트웨어 개발 방법론
- 도메인 전문가와 개발자가 협력해 도메인 모델을 정의하고, 이를 바탕으로 시스템 설계
- 비즈니스 영역을 명확하게 구분, 객체지향적 요소를 활용해 유지보수/확장성을 높임

마이크로서비스 아키텍처(Microservices Architecture, MSA)

- 소규모 서비스 단위로 나누어, 각 서비스가 자체 데이터와 로직을 보유하게 하는 분산 시스템 설계 방식
- 유연한 개발과 배포, 높은 확장성, 장애 격리
- 구현 및 운영의 복잡성과 데이터 일관성 문제

Saga Pattern

- 마이크로서비스 환경에서 분산 트랜잭션을 관리하기 위한 패턴
- 전체 트랜잭션 조율, 실패 시 이전 작업을 보장하는 트랜잭션을 실행
- 보상 로직의 복잡성, 순차 호출로 인한 성능 저하, 모니터링 어려움 등 한계 존재

ROI(Return On Investment)

- 투자 대비 성과를 계량화하는 지표
- $ROI = (\text{투자로 얻은 이득} - \text{투자 비용}) \div \text{투자 비용}$
- 소프트웨어 엔지니어링 맥락에서는 이득은 성능 개선량, 생산성 향상 등으로 계량화되고, 비용은 개발 시간과 노력, 코드라인 수 등으로 산정

관련 연구 및 문제점

기존 연구의 한계

기존 분해 전략	전환 순서 결정 모델 부재	작은 단위 우선순위 모델 부재
• 의미와 구조 기반 메트릭 제시 • 비용 대비 성능 개선을 계량화하지 못함	• 핵심 API를 어떤 순서로 분리해야 효과적인지 구체적 알고리즘이나 수치 모델 부족	• 기존 ROI 연구들은 전체 프로젝트 수준에서 투자 대비 성과만 평가 • 개별 API나 서비스 단위로 우선순위를 정할 수 있는 세부 모델을 제시하지 않음

ROI 기반 MSA 전환 결정 기법

기법 개념 소개

- 기능별 분리 순서를 투자 대비 개선 효과에 맞게 결정
- 각 API 분리 시 얻는 응답시간 절감량과 개발 비용(코드라인 수)을 결합한 ROI 지표 정의
- 높은 개선 효과를 기대할 수 있는 API부터 단계적 분리 결정을 내릴 수 있는 지침 제시

ROI 기반 MSA 전환 결정 기법

실험 시나리오

구현

- 모놀리식 구조와 MSA 구조 구현
- Spring boot 기반으로, MSA는 마이크로 서비스로 분리

측정

- 주요 API를 대상으로 성능 측정 수행
- JMeter로 응답시간 10회 반복 측정 후 개발 코드라인 계산

분석

- 성능 개선도를 개발 비용으로 나눈 ROI 지표 도출 후 분석

ROI 기반 MSA 전환 결정 기법

ROI 지표 산출 방법

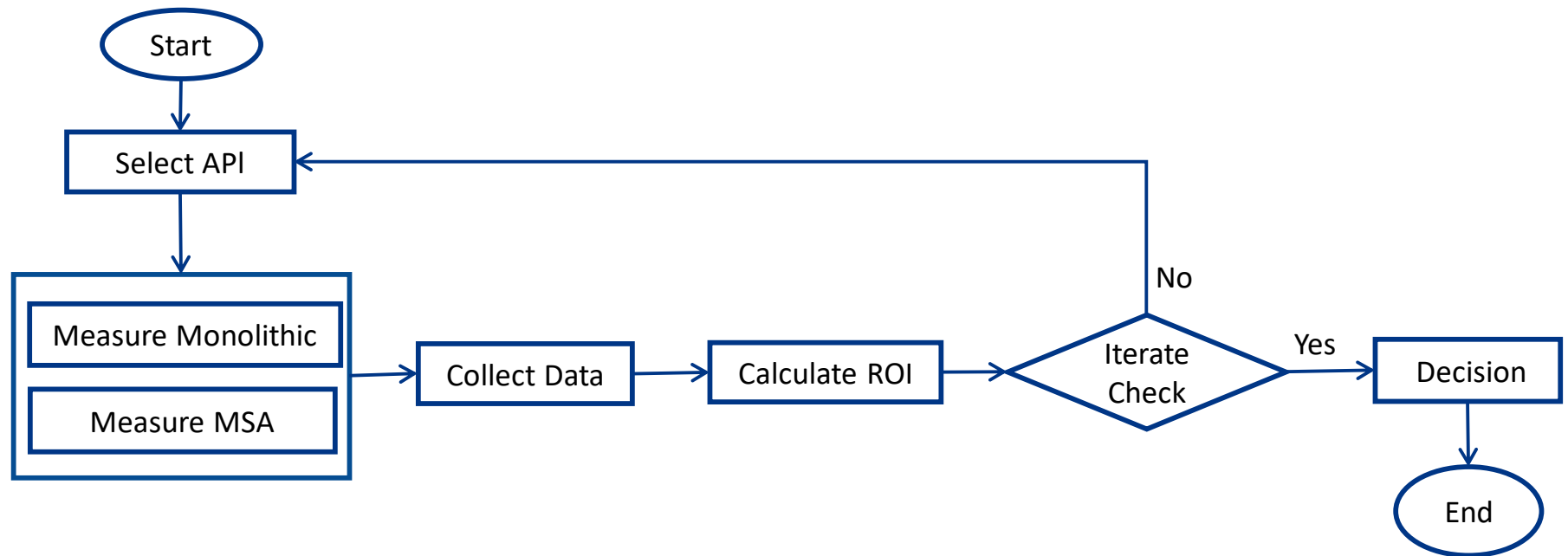
1. 모놀리식 시스템에서 대상 API의 평균 응답시간을 JMeter로 측정하고 기록
2. 동일한 API를 MSA 아키텍처로 분리한 뒤 같은 부하 조건에서 다시 평균 응답시간 측정
3. 두 측정값의 차이인 응답시간 절감량을 각 API 코드라인 수의 차이값으로 나누어 해당 API의 ROI를 계산

계산 수식

- $(\text{DDD 응답시간} - \text{MSA 응답시간}) / (\text{MSA 코드라인} - \text{DDD 코드라인})$

예시) 모놀리식 평균 300ms, MSA 평균 150ms, 코드라인 수의 차이값 100줄인 경우 ROI는 1.5(150ms/100줄)

실험 순서도



향후 연구 일정

1. 실험 준비

- 실험 환경 구성 및 테스트 도구 설정, 데이터베이스 및 네트워크 환경 점검

2. 실험 수행

- Jmeter를 활용하여 각 API별 테스트 후 데이터 수집

3. 분석

- 수집된 데이터를 토대로 계산하여 성능 지표 산출 후 성능 차이 비교

4. 결과 정리

5. 논문 작성

- [1] Yoon, H. et al., “Domain-Driven Design Representation of Monolith Candidate Decompositions Based on Entity Accesses,” Proc. of the Korean Institute of Information Scientists and Engineers, 2025.
- [2] Evans, E., “Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.”, 2003
- [3] Park, J. “마이크로서비스 아키텍처 기반 애플리케이션 프레임워크 연구,” JITA, vol.18, no.4, pp.309–318, 2024.
- [4] Kim, S.W. “마이크로서비스 아키텍처의 빛과 그림자,” Journal of Korea Contents Association, 2024.
- [5] Azure – <https://learn.microsoft.com/en-us/azure/architecture/patterns/saga>
- [6] AWS – <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-choreography.html>
- [7] R. van Solingen, “Measuring the ROI of Software Process Improvement,” IEEE Software, 2004.
- [8] B. Farbey et al., “Evaluation in Software Engineering: ROI, but more than ROI,” 2001.
- [9] Davide Taibi, Kari Systä. “A Decomposition and Metric-Based Evaluation Framework for Microservices.” Proceedings of the 9th International Conference on Cloud Computing and Services Science, May 2019.
- [10] Yuyang Wei, Yijun Yu, Minxue Pan, and Tian Zhang. “A Feature Table approach to decomposing monolithic software systems into microservices.” Proceedings of the 12th Asia-Pacific Symposium on Internetware, May 2021.
- [11] Matt Watson. “Microservices ROI: A Comprehensive Cost-Benefit Analysis.” Fullscale, April 9, 2025.
- [12] Jakub Pepliński. “Monolith to Microservices – Own the Migration Process.” Alokai, June 2024.
- [13] Nuno Alexandre Vieira Santos. “A Complexity Metric for Microservices Architecture Migration.” IEEE International Conference on Software Architecture, March 2020.
- [14] Jonas Fritzsche, Justus Bogner, Alfred Zimmermann, and Stefan Wagner. “Microservices Migration in Industry: Intentions, Strategies, and Challenges.” IEEE International Conference on Software Maintenance and Evolution, October 2019.



감사합니다.
